CineMatch

Movie Recommendation System

电影推荐系统设计与实现

可训练的 ALS-WR 与多信号融合推荐



项目背景与重构目标

维度	原毕业设计	CineMatch
应用架构	SSM 、 JSP 、 MySQL	Vue 3 、 Spring Boot 4 、 Java 21
推荐与运行	Spark ALS 与大数据组件	Java ALS-WR 、请求时个人化
项目维护	历史模块与环境耦合	独立仓库、迁移脚本、自动验收
数据继承	原系统账号与模型	新账号、新数据库、新训练链路

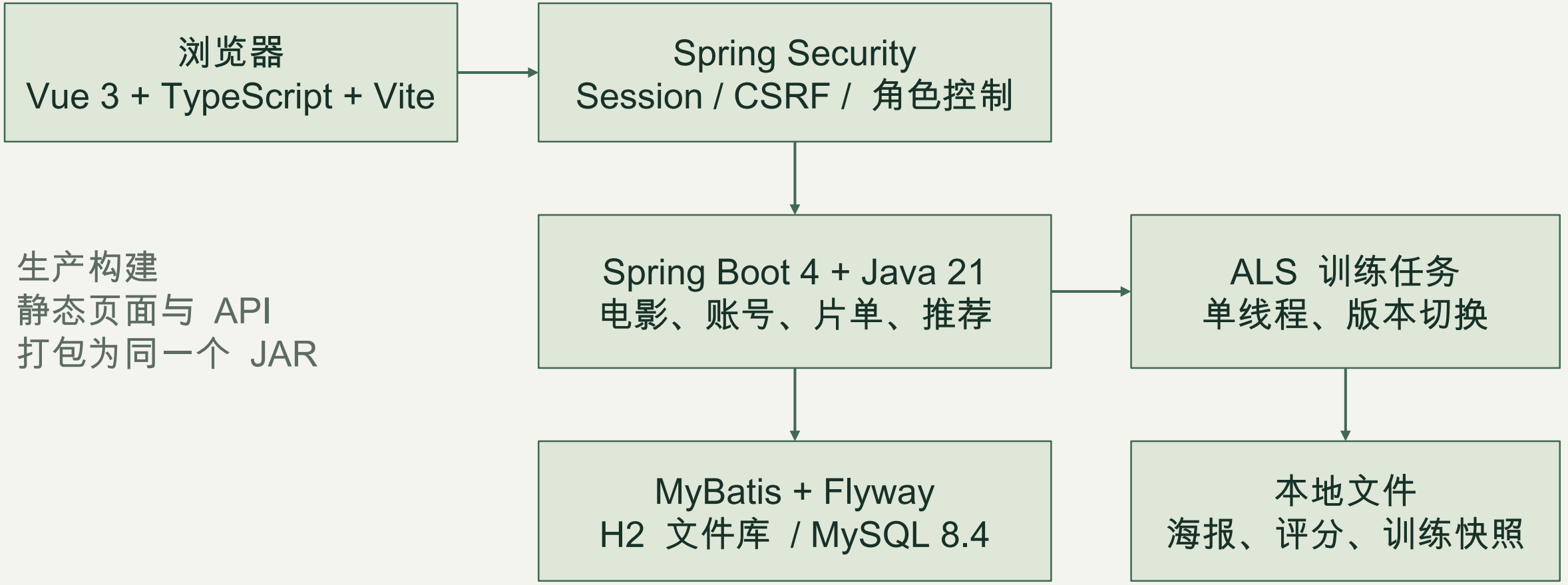
原项目完整保留。新版无需启动旧项目及 Spark / Hadoop 。

系统角色与功能范围

角色	主要入口	已实现的操作
游客	发现、详情、推荐	搜索筛选、电影资料、相似电影、冷启动推荐
登录用户	个人中心、片单、推荐实验室	评分短评、私人合集、偏好、隐藏与恢复
管理员	管理后台	电影与专题维护、模型训练与版本管理 用户概览只读、推荐观察

推荐目标是帮助用户选片。当前不提供电影播放和社交关系功能。

当前系统技术架构


浏览器
Vue 3 + TypeScript + Vite
Spring Security
Session / CSRF / 角色控制
生产构建
静态页面与 API
打包为同一个 JAR
Spring Boot 4 + Java 21
电影、账号、片单、推荐
ALS 训练任务
单线程、版本切换
MyBatis + Flyway
H2 文件库 / MySQL 8.4
本地文件
海报、评分、训练快照

片库与训练评分的两条数据链路

网站片库

923 部

电影资料、来源与海报随仓库提供
导入后用于搜索、详情和内容特征

模型训练源

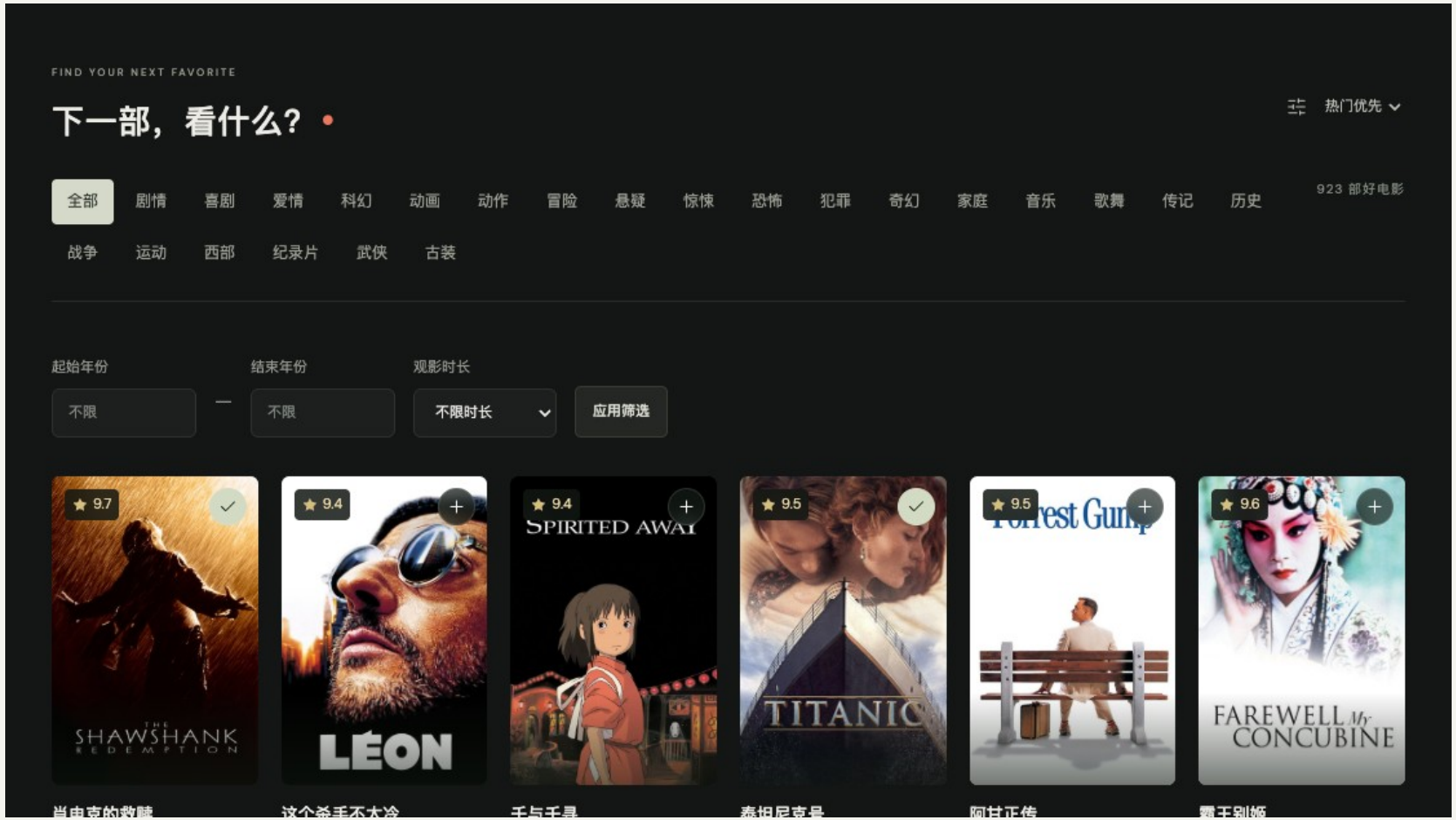
1,000,209 条

6,040 位匿名用户，3,706 部电影
评分源独立下载，不创建本站账号

严格映射： 90 / 923

约 9.75% 片库可参与当前 ALS
其余电影继续使用基础混合推荐

电影发现与组合筛选



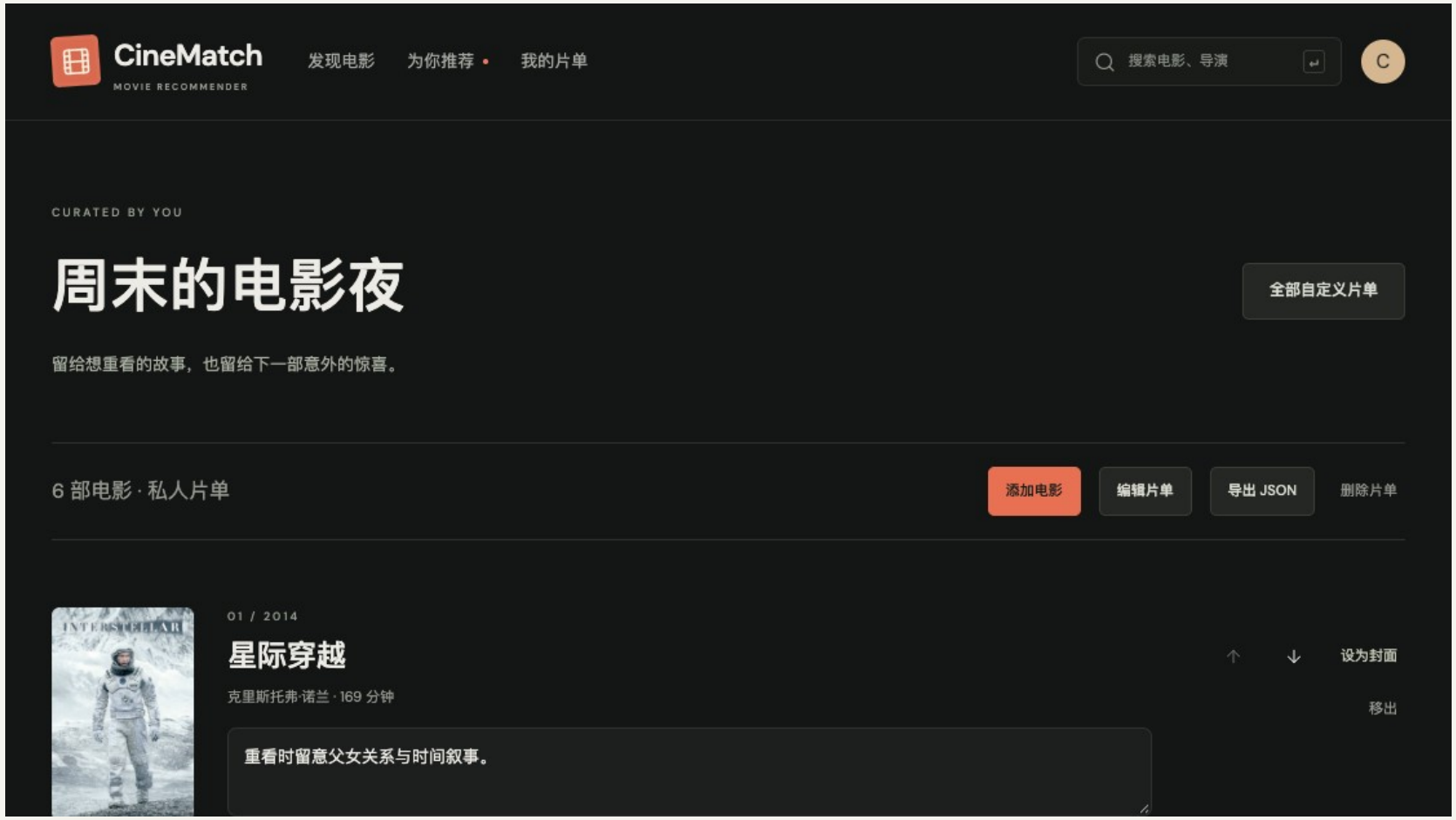
组合筛选

搜索 / 类型 / 年份
时长 / 排序 / 分页

完整片库

923 部电影
来源与历史参考分可追溯

私人片单与观影记录



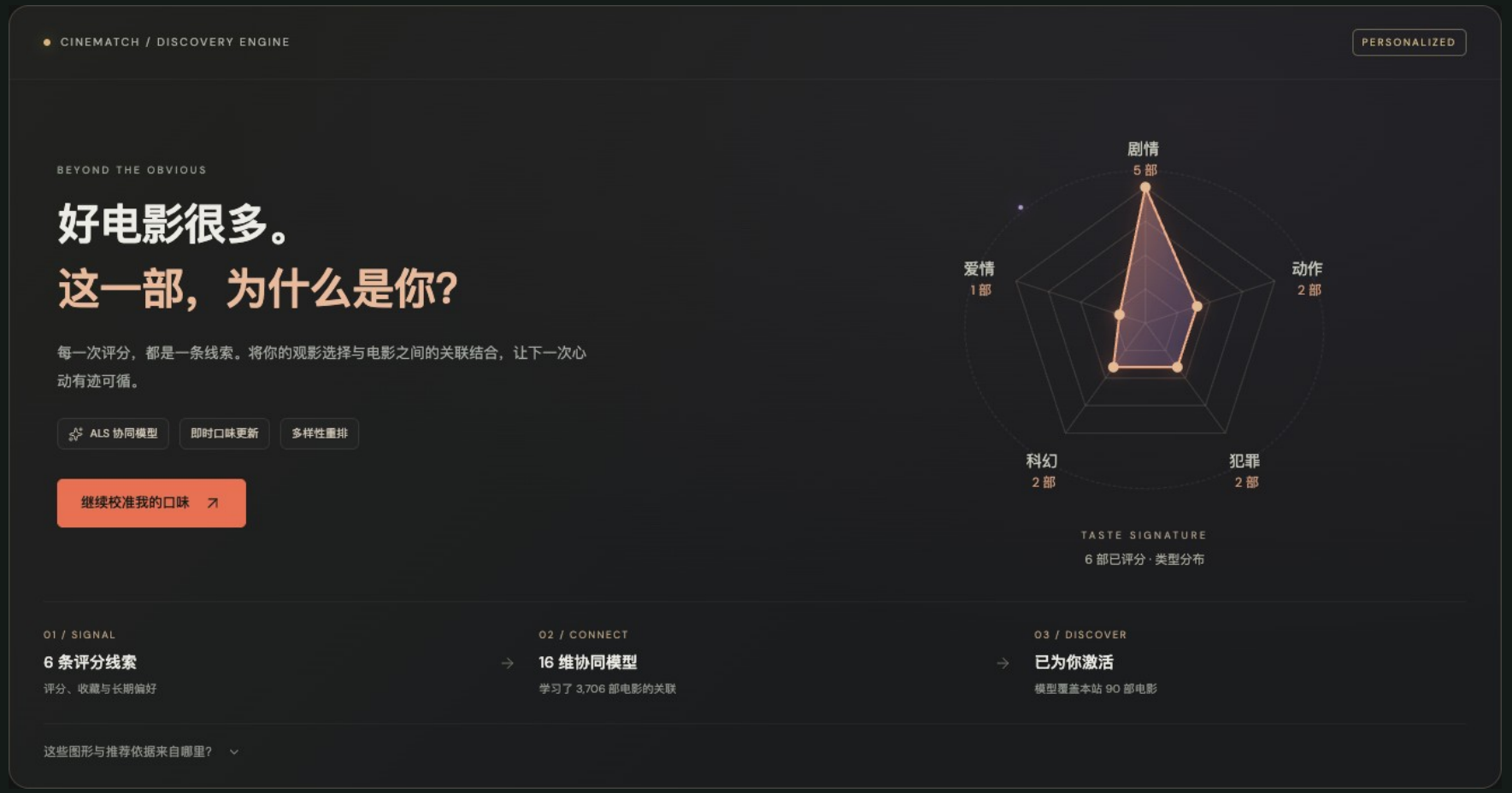
个人记录

想看与看过
评分、短评、类型偏好

自定义片单

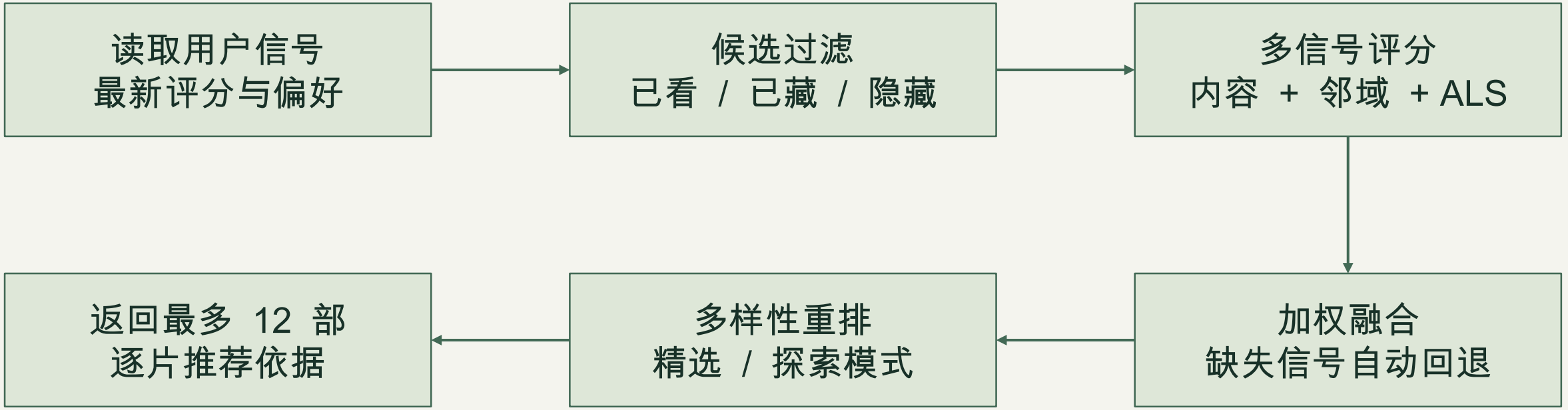
备注、封面、排序
批量操作与 JSON 导出

推荐实验室：评分如何形成个人口味



真实演示账号： 6 部已评分电影。观影指纹展示类型分布，不代表模型准确率。

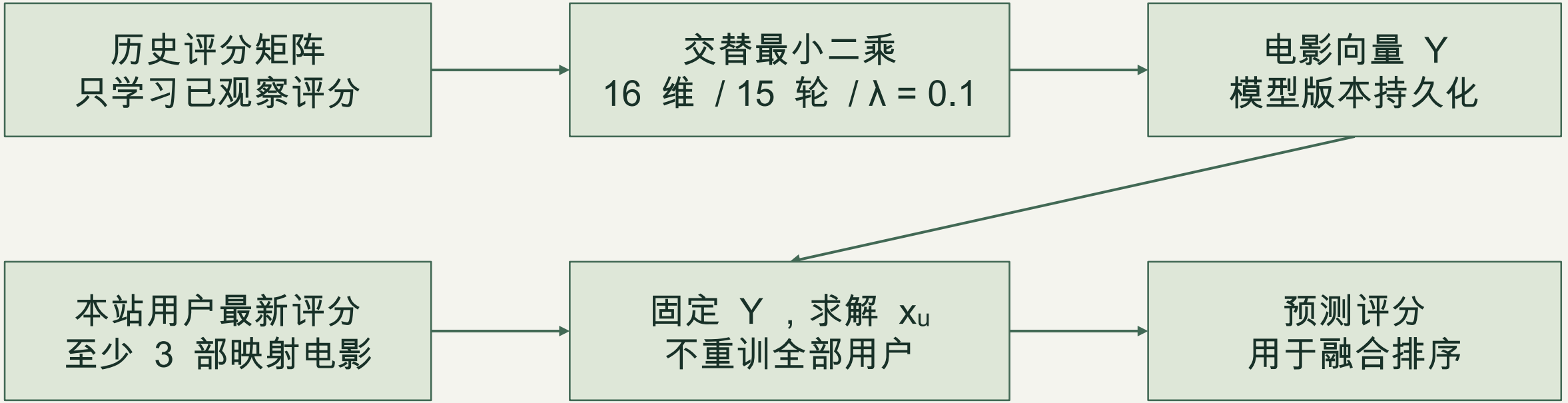
推荐请求的计算流程



探索模式保留相关性门槛；相似度惩罚减少重复；同一导演通常最多 2 部。

ALS-WR ：离线学习与请求时个人化

评分预测 ≈ 用户向量 × 电影向量



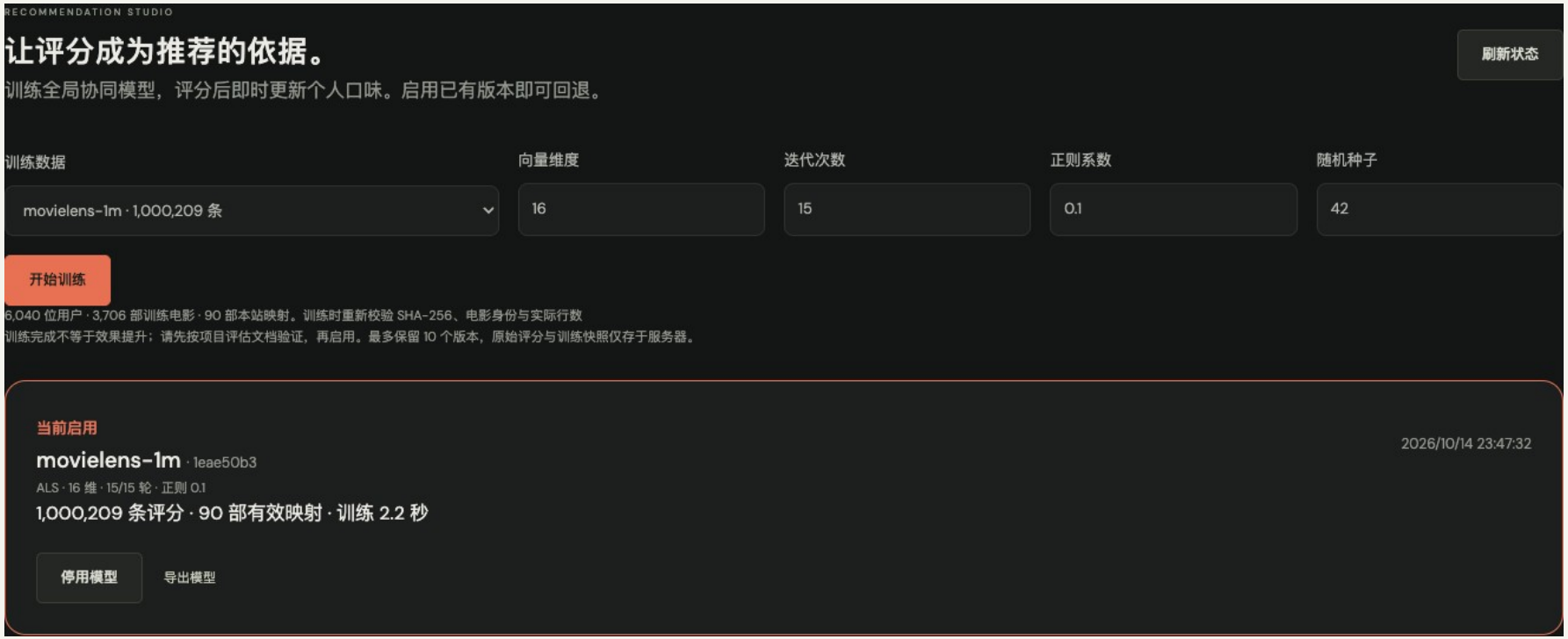
融合得分与冷启动回退

信号	作用	系数
平滑口碑 Q	为低信息用户提供质量基线	0.30
内容偏好 C	类型、导演、国家、年代及负反馈	0.70
邻域协同 N	相似用户的评分偏差预测	0.65
ALS 预测 A	(预测星级 − 3) / 2 ，须有有效映射	0.45

基础分 = 0.30 Q + 0.70 C + 0.65 N

ALS 可用时再加 0.45 A 。无历史且有明确偏好时，冷启动使用 0.85 Q + 0.15 C 。

训练、启用与版本回退



QUEUED / RUNNING / READY 或 FAILED

READY 表示训练成功。效果评估与是否启用由独立流程决定。

按时间隔离训练、验证与测试

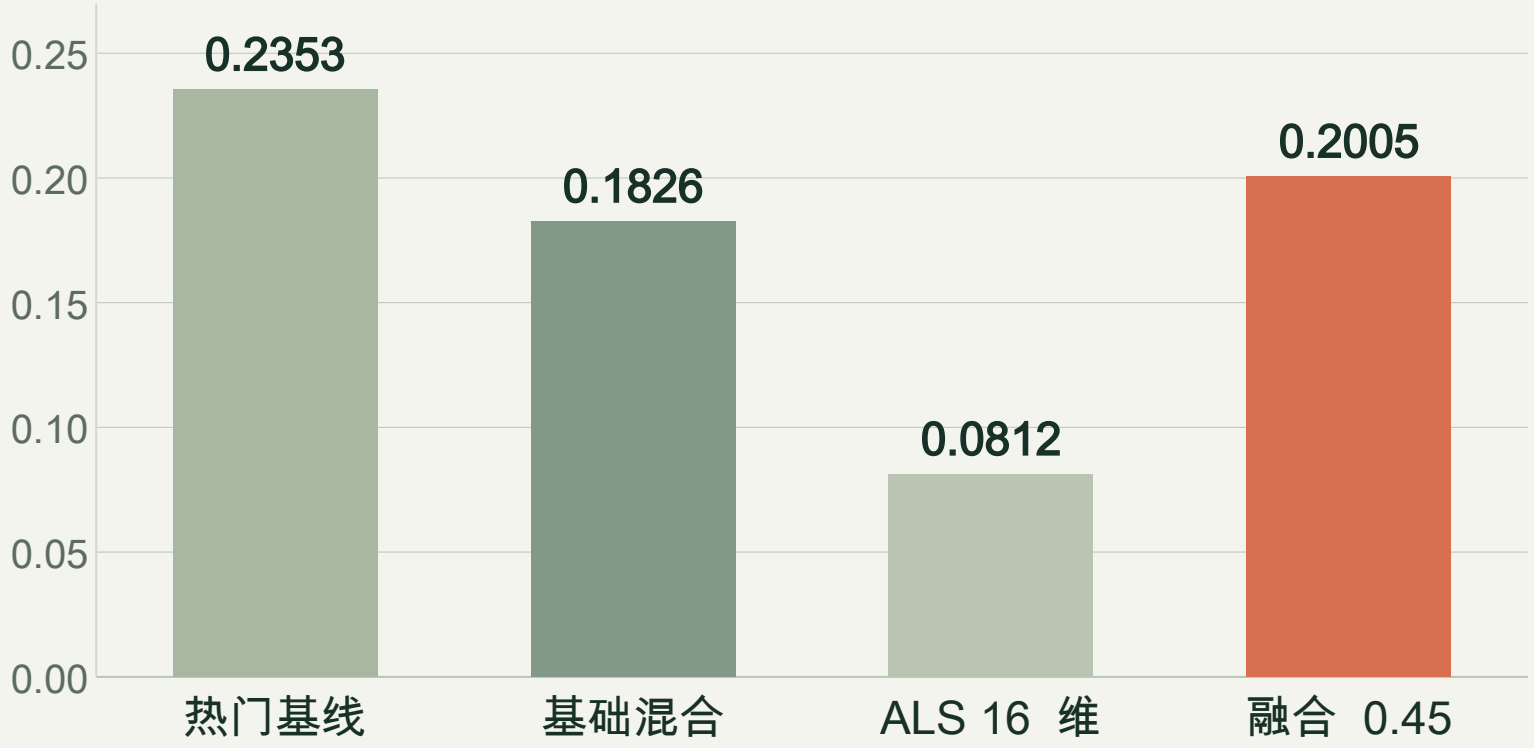
阶段	历史评分	未来评分	用途
验证	700,148	100,020	从 0.45 / 1.5 / 3.0 选择融合权重
测试	800,168	200,041	冻结参数，以前 80% 重新训练

Top 12 ，全候选排序

100 位合格用户；每人至少 20 条历史评分、 5 个未来正样本
测试训练目录 3,662 部电影，再排除各用户已看电影
正样本：未来评分 ≥ 4 星

融合改善原方案，热门基线仍然更强

NDCG@12



+9.84%

相对基础混合 NDCG 提升

Recall@12
4.49% → 5.11%

MovieLens 时间切分测试，100 位用户。该结果不代表 923 部本站片库的在线效果。

效果边界与当前取舍

边界	当前事实	后续改进
覆盖与集中度	目录覆盖 5.60% 降至 4.94%	校准多样性与热门惩罚
应用映射	90 / 923 部可用于当前 ALS	扩充可靠映射及本站评分
人群外推	每人至少 20 条历史记录	单独评估冷启动和少评分用户
线上证据	已有行为观察，尚无随机 A/B	增加曝光量后设计随机实验

训练成功、离线指标提升和线上用户满意度，是三个需要分别验证的结果。

工程验证与部署方式

64

后端测试通过

含真实 MySQL HTTP 验证
另有 2 项可选评估跳过

16

浏览器测试通过

桌面与手机
搜索、片单、推荐、后台

6

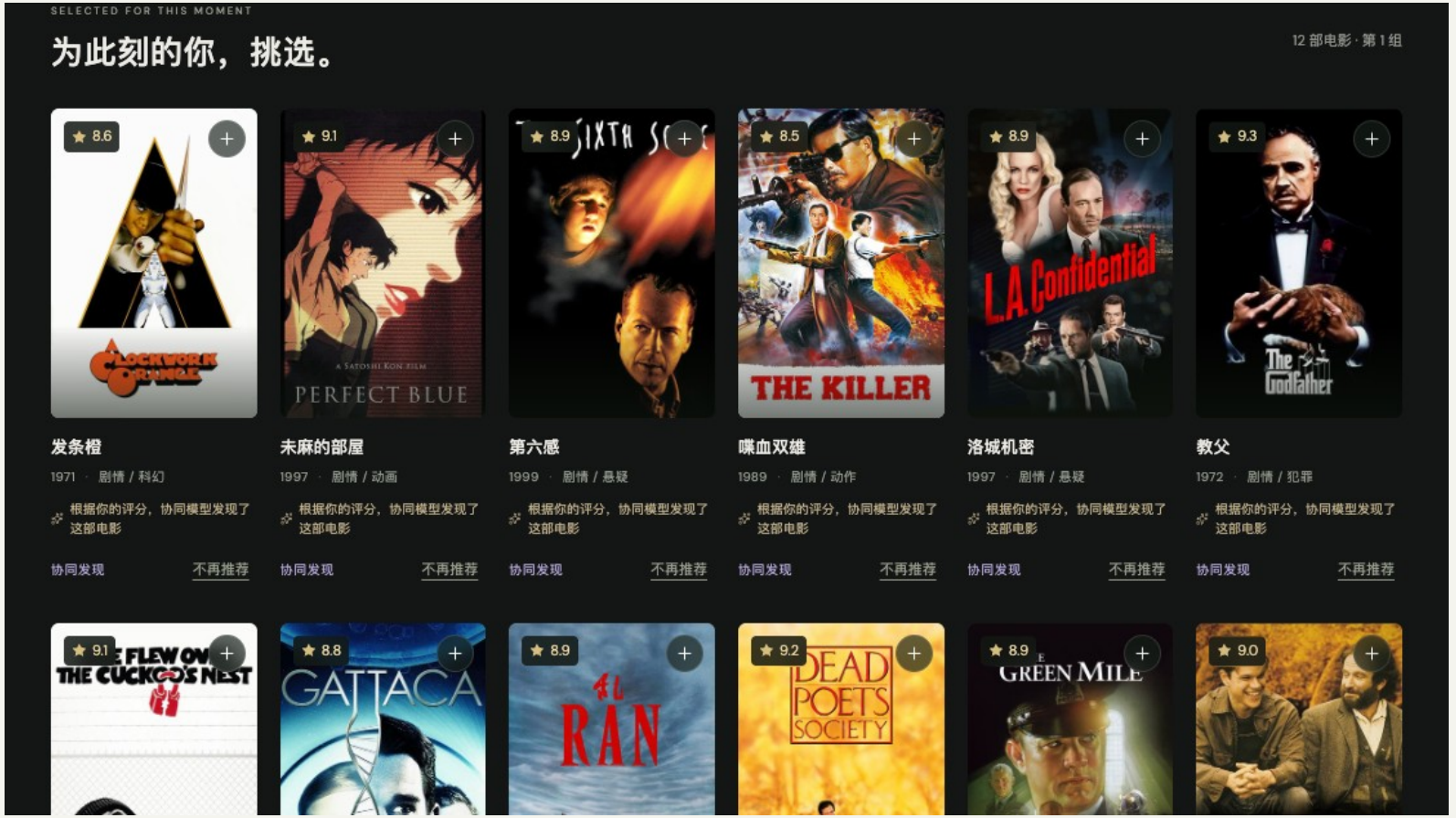
Python 检查通过

片库、数据准备与校验
文件清单及身份映射

部署：单 JAR + H2 ，或 Docker Compose + MySQL 8.4

流水线基准 c3fb14b 。已验证重启持久化，尚未进行最大并发负载测试。

现场演示：评分、推荐与反馈



01 查看口味与模型

6 条评分，16 维活动模型

02 修改一条评分

重新请求推荐，说明即时求解

03 隐藏与探索

观察排除规则
再查看后台版本

项目贡献与下一步

可运行的完整系统

发现电影、私人片单、推荐实验室与后台管理

可训练、可切换、可解释的推荐链路

ALS-WR + 内容与邻域融合，保留评估协议与真实边界

下一步：扩大可靠映射、校准热门偏差、验证冷启动

附录 A ALS-WR 目标与个人向量求解

离线目标（ Ω 为已观察评分 ）

$$\min \sum_{(u,i)\in\Omega} (r_{ui} - x_u^T y_i)^2 + \lambda \left[\sum_u n_u \|x_u\|^2 + \sum_i n_i \|y_i\|^2 \right]$$

固定电影向量，使用当前用户最新评分

$$x_u = \text{solve}\left(\sum_i y_i y_i^T + \lambda n_u I,\ \sum_i r_{ui} y_i \right)$$

默认 rank = 16 ， iterations = 15 ， λ = 0.1 ， seed = 42 。预测星级限制为 0.5–5 。

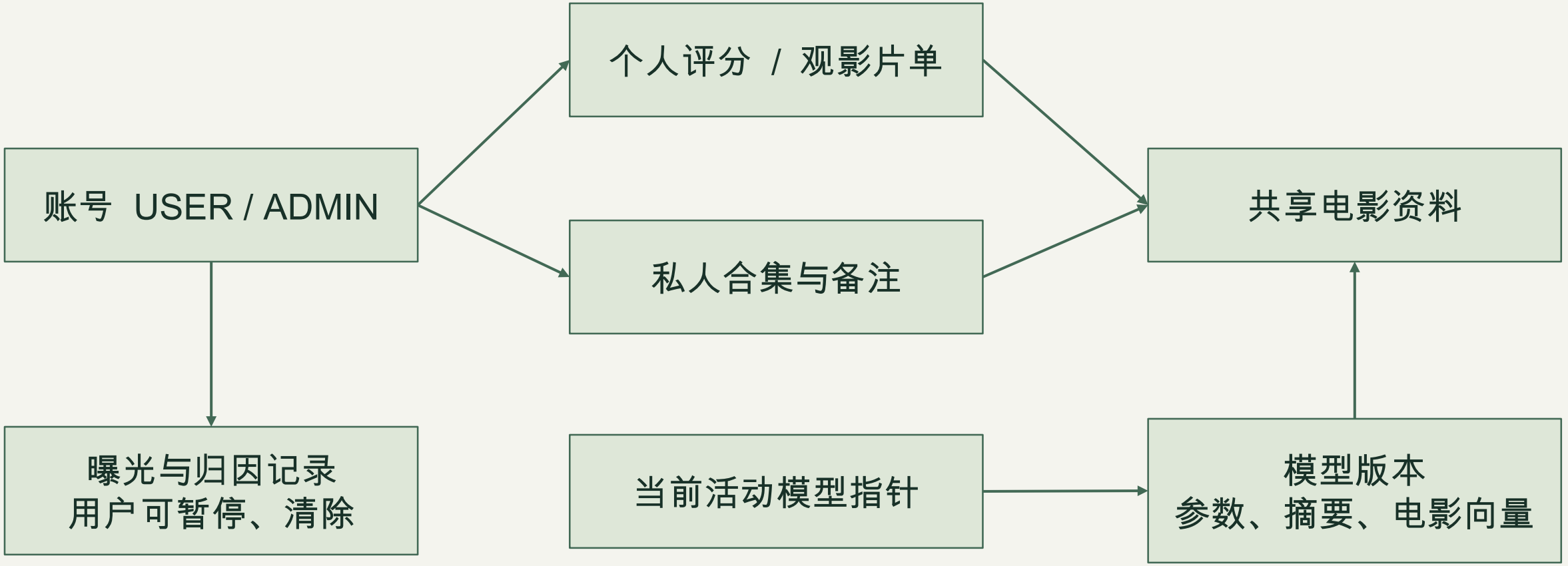
附录 B 完整测试指标

方法	P@12	R@12	NDCG@12	目录覆盖	类型多样性
热门基线	0.2167	0.0533	0.2353	3.17%	0.7847
基础混合	0.1650	0.0449	0.1826	5.60%	0.5604
ALS 1 维	0.0050	0.0005	0.0057	1.39%	0.3784
ALS 16 维	0.0950	0.0280	0.0812	5.79%	0.7342
融合 0.45	0.1850	0.0511	0.2005	4.94%	0.6029

NDCG 绝对增量 95% 区间：[0.0080, 0.0282]

ALS RMSE = 0.8694（ 11,438 条可预测评分， 10 条未见电影评分）

附录 C　数据关系与权限边界



逻辑关系图。 MovieLens 匿名用户与本站账号使用独立命名空间。

附录 D 克隆后的运行与训练顺序

1 启动完整片库

Java 21 、 Node ≥ 22.12 、 Maven ≥ 3.9 、 Python ≥ 3.9
仓库根目录执行： bash scripts/dev.sh --catalog

2 准备独立训练源

下载官方 MovieLens 1M 、 links 与 Wikidata 对照
运行 scripts/prepare_recommender.py ，输出映射与 SHA-256 清单

3 训练并启用

/admin 选择 movielens-1m ，训练完成后检查评估并启用
/picks 为至少 3 部已映射电影评分，参与个人 ALS 计算

附录 E　常见追问

问题	回答要点
为什么不用 Spark ？	当前规模采用单进程 CPU ALS ，降低部署与维护成本。
热门基线更高，融合有什么意义？	相对原方案有提升，但仍需校准偏差并扩展验证。
为什么只映射 90 部？	数据年代与片库重叠有限，且需严格身份和年份校验。
是否实时训练，是否使用大模型？	个人向量按最新评分求解，全局模型离线训练。未使用 LLM 。
3 条评分就能保证准确？	这是参与个人 ALS 的门槛，冷启动效果需要另行评估。

附录 F　资料与复现入口

项目代码

github.com/CodeRayZhang/cinematch-recommender

算法与评估

docs/recommendation.md
docs/recommendation-model-evaluation.md
docs/recommendation-model-test.json

部署与训练

docs/deployment.md · docs/recommendation-models.md